

Hermes Prime Cognitive Infrastructure

Hermes Prime Unified Architecture Brief

Version: 0.1 Status: Foundational Doctrine / System Definition Audience: Human engineers, AI coding agents, orchestration systems, Claude Code, Codex CLI, Gemini CLI, Antigravity, and future automation tooling.

1. What Hermes Is

Hermes Prime is NOT a chatbot. Hermes Prime is NOT a fully autonomous AI employee. Hermes Prime is NOT an unrestricted self-improving agent.

Hermes Prime is:

A governed probabilistic cognitive infrastructure system.

More specifically:

Hermes is a stochastic distributed operating environment where probabilistic models operate inside deterministic trust boundaries.

The architecture treats language models as:

- heuristic reasoning engines
- probabilistic planners
- semantic interpreters
- synthesis systems

—not authoritative execution engines.

The system assumes:

- models can hallucinate
- models can drift
- models can be manipulated
- models can fail unpredictably
- models cannot be fully trusted

Therefore:

Deterministic systems dominate probabilistic systems.

This is the core architectural principle.

2. Core Philosophy

The entire Hermes architecture is based on the following reframing:

Traditional AI Term	Hermes Systems Interpretation
Hallucination	Untrusted stochastic output
Prompt injection	Privilege escalation via data channel
Agent memory	Mutable distributed state
Tool use	RPC to external systems
Multi-agent systems	Distributed coordination
Reasoning	Heuristic planning
Autonomy	Bounded delegated execution

Hermes Prime assumes:

The AI agent problem is fundamentally a distributed systems and governance problem with probabilistic components embedded in the control plane.

This means Hermes Prime prioritizes:

- trust boundaries
- verification
- auditability
- provenance
- reversibility
- bounded execution
- deterministic enforcement
- failure containment

Above:

- raw autonomy
 - unconstrained self-improvement
 - recursive agent swarms
 - uncontrolled memory accumulation
-

3. System Goals

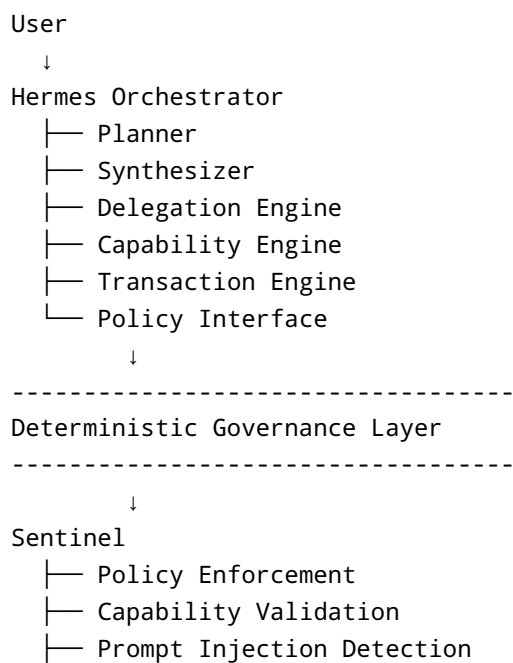
Hermes is designed to:

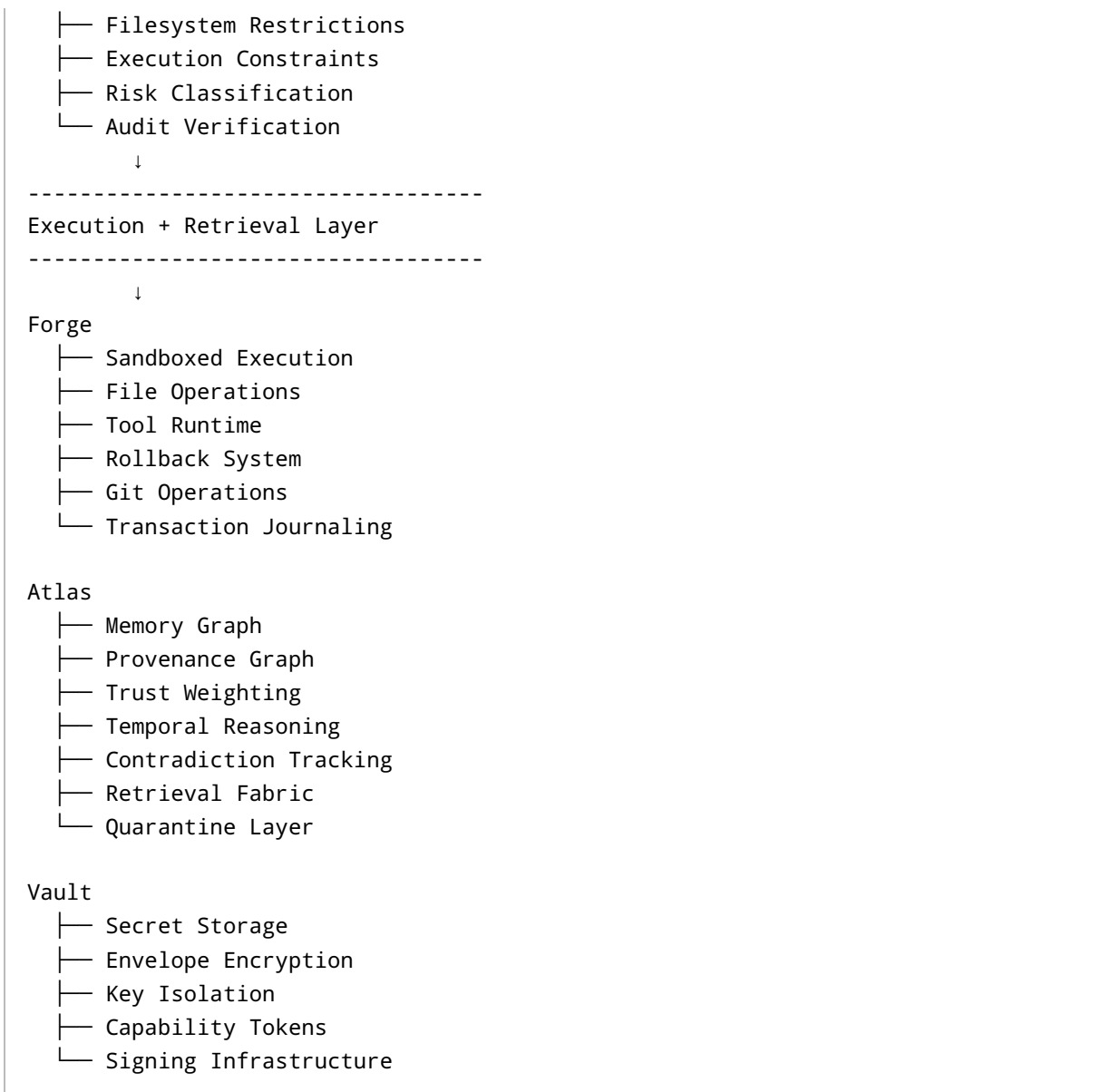
1. Reduce context waste
2. Reduce unsafe autonomous behavior
3. Reduce hallucinated execution
4. Reduce filesystem exploration costs
5. Reduce correlated failure
6. Improve observability
7. Improve reproducibility
8. Improve provenance tracking
9. Improve execution governance
10. Improve long-term maintainability

Hermes Prime prioritizes:

- reliability over novelty
- governance over autonomy
- observability over magic
- modularity over complexity
- bounded systems over unconstrained systems

4. High-Level System Architecture





5. Primary Subsystems

5.1 Hermes

Hermes is the orchestrator.

Responsibilities:

- planning
- synthesis

- delegation
- workflow coordination
- artifact generation
- retrieval requests
- transaction orchestration

Hermes is intentionally treated as:

an untrusted participant inside the system.

Hermes NEVER:

- directly owns secrets
- bypasses Sentinel
- bypasses Vault
- executes unrestricted actions
- writes unrestricted memory
- performs irreversible operations without governance

Hermes is NOT authoritative.

5.2 Sentinel

Sentinel is the deterministic governance and security layer.

Sentinel is one of the most critical components.

Responsibilities:

- deterministic policy enforcement
- prompt injection detection
- capability enforcement
- execution gating
- filesystem restrictions
- trust classification
- risk-tiering
- observability governance
- audit validation

Sentinel combines:

Deterministic Layer

- regex
- schema validation
- AST analysis

- capability verification
- static policies
- execution boundaries
- path restrictions
- signing verification

Probabilistic Layer

- anomaly detection
- semantic risk estimation
- suspicious intent classification
- behavioral analysis

Deterministic systems ALWAYS override probabilistic systems.

5.3 Atlas

Atlas is NOT a vector database. Atlas is NOT authoritative truth.

Atlas is:

a probabilistic epistemic state infrastructure.

Atlas manages:

- memory relationships
- provenance lineage
- trust weighting
- contradiction tracking
- temporal relevance
- memory aging
- retrieval indexing
- graph traversal

Atlas maintains:

Memory Type	Purpose
Episodic Memory	Session history
Semantic Graph	Relationships
Provenance Graph	Source lineage
Trust Layer	Confidence scoring
Temporal Layer	Aging + recency

Memory Type	Purpose
Quarantine Layer	Suspicious memory

Atlas assumes:

- memory can be poisoned
- memory can become stale
- memory can contradict itself
- memory must be versioned
- memory must be attributable

5.4 Forge

Forge is the execution layer.

Forge handles:

- filesystem operations
- shell commands
- git operations
- patch application
- runtime execution
- testing
- rollback
- journaling

Forge MUST:

- operate inside sandboxes
- support rollback where possible
- maintain execution journals
- support transaction semantics
- validate execution schemas

Forge NEVER:

- executes unrestricted commands
 - bypasses Sentinel
 - performs irreversible actions silently
-

5.5 Vault

Vault handles:

- secrets
- signing
- encryption
- token issuance
- capability derivation

Vault uses:

- libsodium
- Argon2id
- envelope encryption
- ephemeral capability tokens
- isolated signing authority

Hermes NEVER directly accesses raw secrets.

Secrets should never enter model context if avoidable.

6. Retrieval Fabric and File Miners

One of Hermes' core innovations is:

retrieval workers instead of brute-force context exploration.

Most coding agents waste enormous token budgets manually searching repositories.

Hermes Prime avoids this using:

- retrieval workers
- file miners
- dependency miners
- log miners
- git miners
- schema miners
- test miners

These workers are:

- bounded
- scoped
- ephemeral

- mostly deterministic
- low-trust
- retrieval-oriented

Workers DO NOT:

- independently execute arbitrary actions
- recursively create more workers
- persist unrestricted memory
- become autonomous planners

Workers retrieve information and return:

- summaries
- graph mappings
- ranked relevance
- structured reports
- dependency relationships

Example:

```
finding:
  file: auth/jwt.ts
  relevance: 0.94
  reason:
    - signs JWT
    - handles refresh
    - imports session store

related:
  - middleware/auth.ts
  - api/login.ts
```

This drastically reduces:

- token usage
- latency
- redundant filesystem traversal
- context bloat

The retrieval layer becomes:

cognitive DMA (direct memory access).

7. Provenance and Docusigner

Hermes Prime implements:

cryptographic provenance infrastructure for cognitive artifacts.

Every important artifact can be:

- signed
- versioned
- attributed
- verified
- audited

Artifacts may include:

- code patches
- plans
- deployment configs
- generated files
- execution requests
- memory writes
- reports

Artifacts contain metadata:

Field	Purpose
artifact_id	unique identifier
creator_model	originating model
generation_time	timestamp
intent_root	linked user intent
verification_state	deterministic checks
review_status	human review state
mutation_history	modification lineage
signature	cryptographic attestation

Signing authority is isolated.

Hermes NEVER directly owns signing keys.

This creates:

- cognitive supply-chain security
 - replayability
 - forensic traceability
 - integrity verification
 - execution lineage
-

8. Intent Provenance

One of Hermes' central ideas:

authorization is NOT intent verification.

Capability systems answer:

“Can this action happen?”

But NOT:

“Did the user actually intend this exact action?”

Hermes attempts to maintain:

- intent lineage
- capability derivation
- action traceability
- execution ancestry

Future architecture direction:

- cryptographic intent chains
- signed action ancestry
- execution provenance graphs

This remains an OPEN PROBLEM.

9. Transaction and Rollback Philosophy

Hermes Prime assumes:

- models can produce incorrect actions
- execution can fail

- plans can drift
- files can be corrupted

Therefore:

Every major operation should support:

- journaling
- rollback
- replay
- verification
- checkpointing
- transaction semantics

Example:

```
begin_transaction
→ plan
→ validate
→ execute
→ verify
→ commit/revert
```

Irreversible actions require:

- stronger verification
- stronger provenance
- stronger authorization

10. Trust and Risk Tiers

Hermes uses risk-tiered execution.

Example:

Risk Level	Example
Low	read-only retrieval
Medium	local file modification
High	git push
Critical	deployment

Risk Level	Example
Irreversible	destructive operations

Higher-risk operations require:

- more verification
- more provenance
- more explicit authorization
- stronger deterministic validation

Consent fatigue is treated as a real systems risk.

Not every action should interrupt the user.

11. Model Routing Philosophy

Hermes does NOT assume one model is sufficient.

Different models have:

- different failure modes
- different strengths
- different reliability profiles

Hermes supports:

- model routing
- model diversity
- quality floors
- validator models
- retrieval models
- local models for simulation
- frontier models for reasoning

Critical operations may require:

- multi-model verification
- deterministic validation
- replay consistency

This reduces:

- correlated stochastic failure
 - model monoculture risk
-

12. Security Philosophy

Hermes Prime follows:

- zero trust
- least privilege
- bounded execution
- deterministic governance
- compartmentalization
- explicit capability derivation

Hermes Prime assumes:

- prompt injection will occur
- memory poisoning will occur
- retrieval contamination will occur
- context drift will occur
- hallucinated actions will occur

The architecture is designed to:

- contain failures
- reduce blast radius
- preserve auditability
- preserve recoverability

—not eliminate all failure.

Perfect safety is assumed impossible.

13. Explicit Non-Goals

Hermes is NOT:

- a fully autonomous employee
- unrestricted AGI infrastructure
- recursive self-improving swarm intelligence
- unrestricted filesystem automation
- unrestricted root-level execution
- authoritative truth infrastructure
- unsandboxed agent autonomy
- unrestricted self-modification

If architecture drift moves toward these:

it is considered architectural failure.

14. Open Problems

The following problems are considered unresolved:

14.1 Intent Provenance

How to verify authentic user intent across long execution chains.

14.2 Temporal Epistemic Integrity

How memory systems maintain coherent belief states over time.

14.3 Verification vs Generation Asymmetry

Generation is cheaper than verification. This is dangerous.

14.4 Correlated Stochastic Failure

Subsystems may fail together due to shared assumptions.

14.5 Observability vs Exposure

Telemetry increases attack surface.

14.6 Human Trust Calibration

Humans often trust systems that merely appear organized.

Current mitigations are acknowledged as incomplete.

15. Architectural Rules

All future development should follow these rules:

1. Deterministic systems dominate probabilistic systems.
2. Secrets never belong to the agent.
3. Retrieval should be cheaper than reasoning.
4. Verification should trend cheaper than generation.

5. Execution must remain bounded.
 6. Observability must be threat-modeled.
 7. Modularity must survive subsystem deletion.
 8. No subsystem should become silently authoritative.
 9. Human override must always exist.
 10. Boring infrastructure is preferred over clever complexity.
-

16. The Deletion Test

Every subsystem must pass:

“Can this subsystem be removed while preserving a minimally functional system?”

If NO:

- coupling is too strong
- architecture is becoming fragile
- boundaries are unclear

Subsystem independence is considered critical.

17. Final Summary

Hermes Prime is best understood as:

a governed probabilistic cognitive infrastructure kernel.

The architecture combines:

- deterministic governance
- bounded execution
- retrieval infrastructure
- provenance systems
- epistemic memory
- transaction semantics
- capability-based security
- cognitive artifact signing
- modular orchestration

The long-term goal is NOT:

- magical autonomy
- unrestricted AI employees

- unconstrained self-improving agents

The goal is:

reliable, observable, recoverable cognitive infrastructure.

Hermes Prime prioritizes:

- trustworthiness over spectacle
- governance over hype
- bounded systems over magical systems
- recoverability over autonomy
- engineering discipline over agent theater

This is the foundational direction of Hermes Prime.